

FinBridge 프로젝트 기획서

github.com/rladntlr/FinBidge

2026-04-26

CONTENTS

FinBridge 기획서	
1. 기획 배경	
2. 해결하고자 한 문제	
3. 핵심 사용자	
금융 IT 운영자	
백엔드 개발자	
시스템 연계 담당자	
4. 핵심 기능	
4.1 다중 프로토콜 병렬 실행	
4.2 통합 결과 반환	
4.3 요청 상태 조회	
4.4 시스템 로그 조회	
4.5 Docker 기반 로컬 검증	
4.6 웹 콘솔	
4.7 실패 프로토콜 재처리	
4.8 인터페이스 등록 및 설정 관리	
4.9 모니터링과 성능관리	
5. 사용자 시나리오	
시나리오 1. 전체 프로토콜 연계 확인	

시나리오 2. 특정 프로토콜만 검증

시나리오 3. 운영자가 요청 ID로 장애 지점 추적

시나리오 4. 실패 프로토콜 재처리

시나리오 5. 인터페이스 비활성화

6. MVP 범위

7. 기대 효과

운영 관점

개발 관점

검증 관점

8. 프로젝트 차별점

8.1 여러 기술을 나열하는 데서 끝나지 않음

8.2 금융권 연계 업무의 현실성을 반영

8.3 로컬 재현성을 중요하게 설계

8.4 리뷰와 테스트를 거쳐 안정성 이슈를 정리

9. 향후 확장 방향

인증과 권한

운영 대시보드

알림

재처리

실제 외부 기관 연계

10. 제출 시 설명 포인트

왜 이 프로젝트를 만들었는가

무엇을 중점적으로 구현했는가

어디까지가 현재 구현이고 어디부터가 확장인가

기술적으로 어떤 점을 보여줄 수 있는가

FinBridge 기획서

프로젝트명: FinBridge

한 줄 소개: 금융권에서 함께 운영되는 SOAP, Kafka, SFTP, Batch, REST 인터페이스를 하나의 요청 흐름으로 실행하고 추적하는 통합관리 시스템

문서 목적: 포트폴리오 제출 시 프로젝트의 문제 정의, 기획 의도, 설계 배경을 설명하기 위한 기획 문서

작성 기준: 현재 구현된 FinBridge 완성본 기준

1. 기획 배경

금융 IT 시스템은 하나의 방식으로만 외부 시스템과 연결되지 않는다. 오래된 레거시 시스템은 SOAP을 사용하고, 실시간 이벤트는 Kafka 같은 메시징 시스템을 거치며, 기관 간 파일 연계는 SFTP로 처리된다. 정산, 마감, 일괄 처리 업무는 Batch로 실행되고, 비교적 최근에 만들어진 서비스는 REST API로 연결된다.

문제는 이 연결 방식들이 모두 따로 관리되기 쉽다는 점이다.

프로토콜별 구현이 흩어지면 운영자는 특정 요청이 어디까지 처리됐는지 한눈에 보기 어렵다. 개발자는 장애가 발생했을 때 SOAP 호출 문제인지, Kafka 발행 문제인지, SFTP 업로드 문제인지, Batch 실행 문제인지 각각의 로그와 결과를 따로 확인해야 한다. 시스템 연계 담당자는 하나의 업무 요청이 여러 연계 채널에 걸쳐 처리되는 상황에서 전체 성공 여부를 빠르게 판단하기 어렵다.

FinBridge는 이 문제를 포트폴리오 프로젝트 범위 안에서 재현하고, 하나의 API와 하나의 추적 모델로 해결하는 것을 목표로 했다.

2. 해결하고자 한 문제

FinBridge가 다루는 핵심 문제는 “다중 인터페이스 연계의 분산된 실행과 추적”이다.

금융 시스템에서는 하나의 업무 요청이 여러 외부 시스템으로 동시에 전달될 수 있다. 예를 들어 고객 거래 데이터가 레거시 시스템에는 SOAP으로 전달되고, 이벤트 시스템에는 Kafka로 발행되며, 파일 기반 기관 연계에는 SFTP로 업로드되고, 내부 정산 프로세스는 Batch로 실행될 수 있다.

이때 필요한 것은 단순히 각 프로토콜을 호출하는 코드가 아니다.

- 요청 단위의 고유 ID
- 요청한 프로토콜 목록
- 프로토콜별 성공, 실패, 타임아웃 결과
- 전체 요청의 최종 상태

- 운영자가 조회할 수 있는 로그
- 로컬에서 재현 가능한 실행 환경

FinBridge는 이 요소들을 하나의 흐름으로 묶어, “다양한 금융 연계 인터페이스를 중앙에서 실행하고 추적하는 시스템”을 구현했다.

기존 상황

- 프로토콜별 구현이 분산
- 요청 단위 추적 ID 없음
- 전체 성공 여부 판단 불가
- 장애 시 각 로그를 개별 확인

FinBridge 해결

- 단일 requestId로 전 프로토콜 추적
- 통합 실행 → 통합 결과 한 번에
- overallStatus 자동 계산
- 운영자가 한 화면에서 장애 파악

3. 핵심 사용자

금융 IT 운영자

운영자는 장애가 발생했을 때 가장 먼저 전체 상태를 확인해야 한다. FinBridge는 요청 ID 기준으로 전체 처리 결과와 프로토콜별 결과를 조회할 수 있게 하여, 어떤 연계 채널에서 문제가 났는지 빠르게 파악하도록 돕는다.

백엔드 개발자

개발자는 여러 프로토콜의 호출 방식과 실패 처리를 한 서비스 안에서 일관되게 다뤄야 한다. FinBridge는 각 프로토콜을 어댑터로 분리하고, 최종 결과 저장과 상태 판단은 `IntegrationService` 에서 중앙화해 유지보수 지점을 명확히 했다.

시스템 연계 담당자

연계 담당자는 외부 기관, 레거시 시스템, 파일 송수신, 이벤트 발행이 하나의 업무 흐름에서 어떻게 연결되는지 설명할 수 있어야 한다. FinBridge는 Docker 기반 로컬 실행과 웹 콘솔을 제공해 실제 흐름을 눈으로 확인할 수 있게 했다.

4. 핵심 기능

4.1 다중 프로토콜 병렬 실행

`POST /api/integrate` 요청을 받으면 사용자가 선택한 SOAP, Kafka, SFTP, Batch, REST 프로토콜을 병렬로 실행한다. 각 프로토콜은 독립 어댑터로 분리되어 있으며, 전체 요청은 하나의 `requestId`로 추적된다.

4.2 통합 결과 반환

각 프로토콜의 처리 결과를 `results`에 모아 반환한다. 전체 결과는 다음 세 가지 상태로 정리된다.

- `ALL_SUCCESS` : 요청한 모든 프로토콜 성공
- `PARTIAL_FAILURE` : 일부 성공, 일부 실패 또는 타임아웃
- `ALL_FAILED` : 모든 프로토콜 실패

응답에는 요청 생성 시각인 `createdAt` 과 전체 처리 완료 시각인 `completedAt` 을 함께 제공한다.

4.3 요청 상태 조회

`GET /api/integrate/{requestId}` 로 이전 요청의 처리 결과를 다시 조회할 수 있다. API 호출 직후 화면에서 확인하거나, 운영자가 특정 요청 ID를 기준으로 추적할 때 사용할 수 있다.

4.4 시스템 로그 조회

`GET /api/logs` 는 요청 시작, 프로토콜별 처리 결과, 최종 완료 이벤트를 조회한다. 프로토콜 필터와 페이지네이션을 지원해 운영자가 필요한 로그만 확인할 수 있다.

4.5 DOCKER 기반 로컬 검증

MySQL, Kafka, SFTP를 Docker Compose로 실행한다. Flyway는 도메인 테이블과 Spring Batch 메타데이터 테이블을 생성한다. Docker E2E 테스트는 실제 MySQL, Kafka, SFTP, Batch 경로를 사용해 로컬에서도 실제 연계 흐름을 검증할 수 있게 한다.

4.6 웹 콘솔

정적 HTML 기반 웹 콘솔을 제공한다. 프로토콜 선택, 통합 요청 실행, 결과 조회, 로그 조회뿐 아니라 인터페이스 등록/설정, 실패 프로토콜 재처리, 모니터링 요약, 프로토콜별 성능 지표를 브라우저에서 확인할 수 있다. 이 화면은 운영 대시보드의 완성형이라기보다, 포트폴리오 제출과 로컬 데모를 위한 기능 확인용 콘솔이다.

4.7 실패 프로토콜 재처리

`POST /api/integrate/{requestId}/retry` 로 이전 요청에서 실패하거나 타임아웃된 프로토콜을 재처리할 수 있다. protocols를 명시하면 지정된 프로토콜만 재처리한다. 생략하면 원본 요청에서 FAILED 또는 TIMEOUT 상태인 프로토콜을 자동으로 선택한다. 재처리 결과는 새 requestId로 독립 추적된다.

4.8 인터페이스 등록 및 설정 관리

`GET/POST/PUT /api/interfaces` 로 프로토콜별 인터페이스를 등록하고 설정을 관리할 수 있다. 각 인터페이스는 이름, endpoint, timeout, enabled 여부를 가진다. enabled=false로 설정된 프로토콜은 통합 요청에서 어댑터 실행 없이 DISABLED 결과로 처리된다. enabled=true인 경우 endpoint와 timeoutMs 설정이 Adapter 실행에 전달된다.

Kafka는 endpoint를 발행 topic으로 사용하고 timeoutMs를 메시지 발행 대기 시간으로 사용한다. SFTP는 endpoint를 업로드 디렉터리로 사용하고 timeoutMs를 session/channel 연결 timeout으로 사용한다. Batch는 endpoint와 interfaceName을 JobParameters로 전달한다. REST와 SOAP은 현재 내부 legacy service 구조에서 endpoint를 호출 대상 식별값으로 로그와 결과 메시지에 반영한다.

웹 콘솔에서 인터페이스 목록 조회, 등록, enabled 여부 변경, 설정 수정이 가능하다.

4.9 모니터링과 성능관리

`GET /api/monitoring/summary` 로 전체 요청 수, 처리 중 요청 수, 완료 요청 수, 전체 성공/부분 실패/전체 실패 건수, 전체 로그 수를 조회할 수 있다.

`GET /api/performance/protocols` 는 프로토콜별 처리 건수, 성공/실패/타임아웃 건수, 성공률, 평균 실행 시간을 제공한다. 웹 콘솔에서도 운영자가 이 지표를 확인할 수 있어, 실행 결과뿐 아니라 전체 처리 현황과 프로토콜별 품질을 함께 볼 수 있다.

5. 사용자 시나리오

시나리오 1. 전체 프로토콜 연계 확인

1. 사용자가 웹 콘솔 또는 API 클라이언트에서 SOAP, Kafka, SFTP, Batch, REST를 모두 선택한다.
2. `POST /api/integrate` 요청을 보낸다.
3. FinBridge가 5개 어댑터를 병렬로 실행한다.
4. 사용자는 `requestId` , 전체 상태, 프로토콜별 결과를 즉시 확인한다.
5. 필요하다면 `GET /api/logs` 에서 처리 로그를 확인한다.

시나리오 2. 특정 프로토콜만 검증

1. 개발자가 SFTP만 선택해 요청한다.
2. FinBridge는 SFTP 어댑터만 실행한다.
3. SFTP 컨테이너의 `/upload` 경로에 파일이 생성된다.
4. 개발자는 API 응답과 컨테이너 파일을 함께 확인해 연계 경로를 검증한다.

시나리오 3. 운영자가 요청 ID로 장애 지점 추적

1. 운영자가 실패한 요청의 `requestId` 를 전달받는다.
2. `GET /api/integrate/{requestId}` 로 전체 결과를 조회한다.
3. 실패한 프로토콜과 응답 메시지를 확인한다.
4. `GET /api/logs?protocol=...` 로 해당 프로토콜 로그를 좁혀 본다.

시나리오 4. 실패 프로토콜 재처리

1. 운영자가 `requestId`로 처리 결과를 조회해 SFTP가 FAILED 상태임을 확인한다.
2. `POST /api/integrate/{requestId}/retry` 를 호출한다. `protocols`를 생략하면 FAILED 프로토콜이 자동 선택된다.
3. 재처리 결과가 새 `requestId`로 반환된다.
4. 운영자는 `originalRequestId`와 `retryRequestId`를 함께 기록해 이력을 연결한다.

시나리오 5. 인터페이스 비활성화

1. 연계 담당자가 특정 외부 시스템의 점검으로 SOAP 연계를 일시 중단해야 한다.
2. `PUT /api/interfaces/{id}` 로 SOAP 인터페이스의 `enabled`를 `false`로 변경한다.
3. 이후 들어오는 통합 요청에서 SOAP은 어댑터 실행 없이 DISABLED로 처리된다.
4. 점검이 끝나면 `enabled`를 다시 `true`로 변경해 복구한다.

6. MVP 범위

현재 FinBridge의 MVP는 “로컬에서 실제로 실행 가능한 다중 프로토콜 통합관리 시스템”이다.

구현된 범위:

영역	구현 항목
API	통합 실행, 상태 조회, 로그 조회, 재처리, 인터페이스 등록/수정/조회, 모니터링 요약, 성능관리
어댑터	SOAP, Kafka, SFTP, Batch, REST
인프라	MySQL + JPA, Flyway 스키마 관리, Docker Compose, Kafka DLT, Spring Batch 메타데이터
설정 관리	프로토콜 enabled/disabled 실행 반영, endpoint · timeoutMs Adapter 전달
보안	SFTP <code>StrictHostKeyChecking=yes</code> , Docker host key 고정
테스트	단위 테스트, H2 기반 통합 테스트, Docker E2E 테스트
UI	실행 · 조회 · 인터페이스 관리 · 모니터링 · 재처리 · 성능관리 웹 콘솔

MVP에서 제외한 범위:

- 사용자 인증과 권한 관리
- 운영용 대시보드 차트
- Slack, Email 등 알림 연동
- 실제 외부 금융기관 시스템 연동
- 운영 배포 환경 구성

7. 기대 효과

운영 관점

프로토콜별 결과와 전체 상태를 같은 요청 ID로 추적할 수 있다. 장애 발생 시 어떤 연계 채널이 실패했는지 빠르게 확인할 수 있다.

개발 관점

어댑터별 책임과 통합 서비스의 책임이 분리된다. 새 프로토콜을 추가하더라도 기존 흐름을 크게 바꾸지 않고 어댑터와 결과 매핑을 확장할 수 있다.

검증 관점

단순 Mock 데모가 아니라 Docker 기반으로 MySQL, Kafka, SFTP, Batch 실행까지 확인할 수 있다. 포트폴리오 평가자가 로컬에서 직접 실행해 기능을 검증할 수 있다.

8. 프로젝트 차별점

8.1 여러 기술을 나열하는 데서 끝나지 않음

FinBridge는 SOAP, Kafka, SFTP, Batch, REST를 각각 따로 보여주는 프로젝트가 아니다. 하나의 요청이 여러 프로토콜을 병렬로 실행하고, 하나의 응답과 로그 모델로 모이는 흐름을 구현했다.

8.2 금융권 연계 업무의 현실성을 반영

금융권 시스템은 새로운 REST API만으로 구성되지 않는다. 레거시 SOAP, 파일 기반 SFTP, 이벤트 기반 Kafka, 일괄 처리 Batch가 함께 존재한다. FinBridge는 이 혼합 환경을 의도적으로 다룬다.

8.3 로컬 재현성을 중요하게 설계

Docker Compose로 MySQL, Kafka, SFTP를 실행하고, Flyway로 스키마를 관리한다. SFTP host key도 고정해 매번 known_hosts 충돌을 해결해야 하는 문제를 줄였다.

8.4 리뷰와 테스트를 거쳐 안정성 이슈를 정리

Batch 메타데이터 테이블, Kafka advertised listener, SFTP host key, 로그 페이지네이션, 프로토콜 검증, timeout 처리, Adapter 결과 저장 책임 중앙화 등 런타임에서 문제가 될 수 있는 항목을 리뷰와 테스트를 통해 정리했다.

9. 향후 확장 방향

인증과 권한

현재 API는 로컬 데모 기준으로 열려 있다. 운영 환경에서는 사용자 인증, 관리자 권한, 요청자별 조회 범위 제한이 필요하다.

운영 대시보드

현재 웹 콘솔은 프로토콜별 성공률과 평균 실행 시간을 테이블로 제공한다. 운영 대시보드로 확장하려면 기간별 성공률 추이, p95/p99 latency, SLA 초과 건수, 최근 장애 현황, 알림 상태 등을 차트로 시각화할 수 있다.

알림

특정 프로토콜 실패, 전체 실패, timeout 발생 시 Slack, Email, SMS 등으로 알림을 보낼 수 있다.

재처리

실패 프로토콜 단위 재처리 API는 구현되어 있다. 운영 확장 시에는 재처리 이력 연결 조회, 재처리 횟수 제한, 자동 재처리 스케줄링을 추가할 수 있다.

실제 외부 기관 연계

현재 REST와 SOAP은 프로젝트 내부의 legacy service를 호출하는 구조다. 운영 확장 시에는 외부 end-point 설정, 인증서, 네트워크 보안, 기관별 전문 변환 규칙이 필요하다.

10. 제출 시 설명 포인트

왜 이 프로젝트를 만들었는가

금융 IT에서는 여러 인터페이스가 동시에 존재하고, 각 연계 상태를 한 곳에서 추적하는 일이 중요하다. FinBridge는 이 상황을 작은 범위로 재현한 통합관리 시스템이다.

무엇을 중점적으로 구현했는가

단순 CRUD가 아니라 다중 프로토콜 병렬 실행, 결과 통합, 로그 추적, Docker 기반 실제 검증에 집중했다.

어디까지가 현재 구현이고 어디부터가 확장인가

현재 구현은 로컬에서 실행 가능한 MVP다. 실패 프로토콜 재처리까지는 구현되어 있으며, 인증, 운영용 대시보드 고도화, 알림, 실제 기관 연계는 향후 확장 범위로 명확히 분리했다.

기술적으로 어떤 점을 보여줄 수 있는가

Spring Boot 기반 API 설계, JPA/Flyway 스키마 관리, Kafka 메시징, SFTP 파일 전송, Spring Batch 실행, SOAP/REST 어댑터 구조, CompletableFuture 병렬 처리, Docker E2E 테스트를 하나의 프로젝트 안에서 설명할 수 있다. 재처리 API 설계(원본 requestId 기반 FAILED/TIMEOUT 자동 선택, 새 requestId 독립 추적), 인터페이스 설정 도메인(DB 기반 enabled/disabled, 어댑터 실행 전 설정값 반영)도 함께 설명할 수 있다.